

LABORATORY OBSERVATION / RECORD

Course:	Full Stack Web Development	Course Code:	23CM4121
Experiment No.:	Observation Task 5	Date:	21-09-2026
Student Name:	M. Naga Doleswari Aruna Devi	Roll No.:	A24126552267

1. TITLE

Design and Develop an Interactive Product Calculator Webpage Using HTML, CSS and JavaScript

2. AIM

To design and develop a webpage titled “Interactive Product Calculator” that accepts a product name, quantity and price from the user, and using JavaScript functions and event listeners, dynamically calculates and displays the total price whenever the quantity or price changes — without refreshing the page — while validating the entered data.

3. OBJECTIVE

The objectives of this experiment are:

- To understand and use JavaScript event listeners (click, input, change) to detect user actions on a form.
- To read and process form input values using the Document Object Model (DOM).
- To perform arithmetic calculations dynamically in JavaScript based on live user input.
- To validate user input (empty product name, non-positive quantity, negative price) before performing a calculation.
- To update page content dynamically using innerHTML / textContent without reloading the page.
- To design a clean, responsive interface for the calculator using CSS.

4. THEORY

JavaScript allows a webpage to respond to user actions in real time without reloading the page. This is achieved using event listeners — functions registered on DOM elements with `addEventListener()` — that run whenever a specified event (such as `click`, `input` or `change`) occurs on that element.

The values entered by the user are read using `document.getElementById()` and the `.value` property. Numeric fields are converted from strings using `Number()`, and `isNaN()` is used to check whether a value is a valid number. Combining these checks with simple comparisons (e.g. `quantity > 0`, `price ≥ 0`) allows the form to be validated before any calculation is performed.

Once the input is validated, the total price is computed as `quantity × price`, and the result is written back into the page by setting the `innerHTML` of a result element, updating the visible content instantly and dynamically.

5. ALGORITHM / PROCEDURE

- 1 Create three files: `index.html`, `style.css` and `script.js`.

- 2 In index.html, build a container with a heading, a form containing Product Name, Quantity and Price inputs, a "Calculate total" button, and an empty result heading.
- 3 Link style.css for styling and script.js for the calculation logic.
- 4 In style.css, style the body, container, labels, inputs, button and result area.
- 5 In script.js, select the productName, quantity, price and result elements using getElementById.
- 6 Define a calculateTotal() function that reads the current values of the three inputs.
- 7 Inside calculateTotal(), validate the input: Product Name must not be empty; Quantity must be a number greater than 0; Price must be a number that is not negative.
- 8 If validation fails, display "Please enter valid details." in the result area.
- 9 If validation passes, compute total = quantity × price and display the Product, Quantity and Total Price.
- 10 Attach calculateTotal() to the button's click event, and to the input and change events of all three fields, so the total updates live as the user types.
- 11 Save the files and open index.html in a browser.
- 12 Test the page with valid and invalid combinations of inputs and record the observations against the expected results.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Interactive Product Calculator</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
<h1>Interactive Product Calculator</h1>
</header>

<div class="container">
<h2>the Product calculator</h2>

<label for="productName">Product Name:</label>
<input type="text" id="productName" placeholder="Enter product name">

<label for="quantity">Quantity:</label>
<input type="number" id="quantity" placeholder="Enter quantity" min="1">

<label for="price">Price:</label>
<input type="number" id="price" placeholder="Enter price" min="0">

<button onclick="calculateTotal()">Calculate total</button>

<h2 id="result"></h2>
</div>

<script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
font-family: Arial, sans-serif;
background-color: #f2f2f2;
```

```

}

.container {
width: 400px;
margin: 100px auto;
padding: 25px;
background-color: white;
border-radius: 10px;
box-shadow: 0 0 10px gray;
}

h1 {
text-align: center;
}

label {
display: block;
margin-top: 15px;
}

input {
width: 95%;
padding: 10px;
margin-top: 5px;
}

button {
margin-top: 20px;
padding: 10px 20px;
cursor: pointer;
}

#result {
margin-top: 20px;
}

```

script.js

```

const nameInput = document.getElementById('productName');
const qtyInput = document.getElementById('quantity');
const priceInput = document.getElementById('price');
const resultEl = document.getElementById('result');

function calculateTotal() {
const name = nameInput.value.trim();
const qty = Number(qtyInput.value);
const price = Number(priceInput.value);

const isValid =
name !== '' &&
!isNaN(qty) && qty > 0 &&
!isNaN(price) && price >= 0;

if (!isValid) {
resultEl.textContent = 'Please enter valid details.';
return;
}

const total = qty * price;
resultEl.innerHTML =
'Product: ' + name + '<br>' +
'Quantity: ' + qty + '<br>' +
'Total Price: \u20B9' + total;
}

// Recalculate live as the user types or changes a field
[nameInput, qtyInput, priceInput].forEach(el => {
el.addEventListener('input', calculateTotal);
el.addEventListener('change', calculateTotal);
});

```

```
});
```

7. CODE EXPLANATION

Tag / Statement	Description
<link rel="stylesheet" href="style.css">	Links the external CSS file used to style the calculator.
<input id="productName/quantity/price">	Form controls used to capture the product name, quantity and price entered by the user.
document.getElementById()	Selects the input and result elements so their values can be read or updated from script.js.
addEventListener('input'/'change', ...)	Registers calculateTotal() to run automatically whenever a field's value changes, enabling live recalculation without a page refresh.
Number(), isNaN()	Convert the text entered by the user into numbers and check whether the conversion produced a valid number.
Validation condition (name, qty > 0, price >= 0)	Ensures the product name is not empty and that quantity/price are sensible numbers before calculating the total.
resultEl.textContent = 'Please enter valid details.'	Displays a validation error message when any of the input checks fail.
total = qty * price	Computes the total price by multiplying the validated quantity and price.
resultEl.innerHTML = ...	Dynamically updates the result heading with the Product, Quantity and Total Price, without reloading the page.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and tested with the following product name, quantity and price combinations, each verified against the expected result:

Test Case	Product Name	Quantity	Price	Expected Result	Status
TC-01	GT650	20	450000	Total Price: ₹9000000	Pass
TC-02	Pen	5	10	Total Price: ₹50	Pass
TC-03	Laptop	2	50000	Total Price: ₹100000	Pass
TC-04	Book	3	250	Total Price: ₹750	Pass
TC-05	Mobile	1	20000	Total Price: ₹20000	Pass
TC-06	(empty)	2	100	Please enter valid details.	Pass
TC-07	Pen	0	10	Please enter valid details.	Pass
TC-08	Pen	5	-10	Please enter valid details.	Pass
TC-09	(empty)	0	0	Please enter valid details.	Pass

9. OUTPUT

Actual output produced by running the program above in a browser (shown here for the TC-01 input values):

The image shows a web form titled "Interactive Product Calculator" with the subtitle "the Product calculator". It contains three input fields: "Product Name:" with the value "GT650", "Quantity:" with the value "20", and "Price:" with the value "450000". Below these fields is a blue button labeled "Calculate total". At the bottom of the form, the results are displayed: "Product: GT650", "Quantity: 20", and "Total Price: ₹9000000".

Field	Value
Product Name	GT650
Quantity	20
Price	450000
Product	GT650
Quantity	20
Total Price	₹9000000

Fig. 9.1 — Interactive Product Calculator showing a valid calculation

10. RESULT

Thus, a webpage titled “Interactive Product Calculator” was successfully designed and developed using HTML, CSS and JavaScript. JavaScript functions and event listeners were used to dynamically calculate and display the total price whenever the quantity or price changed, without refreshing the page. Input validation for an empty product name, a non-positive quantity and a negative price was implemented and verified through browser testing, matching the expected results shown above.

1. TITLE

Node.js File Management Using the File System (fs) Module

2. AIM

To develop a Node.js application that accepts a filename and content from the user and, using the File System (fs) module, creates/writes the file, reads its contents, appends additional content and displays the final contents of the file — while validating the entered data.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the Node.js runtime and how built-in modules are loaded using `require()`.
- To create and write a file using the File System module (`writeFileSync()`).
- To read the contents of a file (`readFileSync()`) and display them on the console.
- To append additional content to an existing file (`appendFileSync()`) without losing the old content.
- To accept the filename and content from the user at run time using the `readline` module.
- To validate user input (empty filename, empty content) and display suitable error messages.
- To observe the difference between overwriting a file (`write`) and appending to a file (`append`).

4. THEORY

Node.js is a JavaScript runtime that executes JavaScript outside the browser. Its built-in modules are loaded with `require()`. The File System module (fs) allows a program to work with files on the computer — to create, read, update and delete them.

The main methods used in this experiment are: `writeFileSync(file, data)` creates a new file, or overwrites the file if it already exists; `readFileSync(file, 'utf8')` returns the entire content of the file as a string; `appendFileSync(file, data)` adds data at the end of the file and keeps the existing content; and `existsSync(file)` checks whether a file is already present. The `sync` versions wait until the operation is completed, which keeps the program simple; their asynchronous counterparts (`writeFile`, `readFile`, `appendFile`) use callbacks or promises.

The filename and content are accepted from the keyboard using the `readline` module, which reads lines from standard input (`process.stdin`). The input is validated with `trim()` so that an empty (or blank) filename or content is rejected before any file operation is performed, and the results are displayed on the console using `console.log()`.

5. ALGORITHM / PROCEDURE

- 1 Create a project folder and a file named app.js.
- 2 Import the fs and readline modules using require().
- 3 Create a readline interface and define a helper function ask() that displays a prompt and returns the line typed by the user.
- 4 Accept the filename from the user. If it is empty, display "Error: Filename is required" and stop.
- 5 Check whether the file already exists using fs.existsSync().
- 6 Accept the content from the user. If the content is empty and the file does not exist, display "Error: Content cannot be empty" and stop. If the content is empty but the file already exists, keep the existing file unchanged.
- 7 If content is entered, create/write the file using fs.writeFileSync() (a new file is created; an existing file is overwritten) and display a success message.
- 8 Read the file using fs.readFileSync() and display its contents.
- 9 Accept additional content from the user. If it is not empty, append it using fs.appendFileSync() and display a success message.
- 10 Read the file again with fs.readFileSync() and display the final contents.
- 11 Close the readline interface. Save the file and run it using the command: node app.js
- 12 Test the program with valid and invalid combinations of inputs and record the observations against the expected results.

6. PROGRAM

app.js

```
// File Management using the Node.js File System (fs) module
const fs = require('fs');
const readline = require('readline');

const rl = readline.createInterface({ input: process.stdin });
const lines = rl[Symbol.asyncIterator]();

// Show a prompt and wait for the user to type one line
async function ask(question) {
  process.stdout.write(question);
  const { value } = await lines.next();
  return value === undefined ? '' : value;
}

async function main() {
  console.log('===== Node.js File Management =====');

  // 1. Accept the filename and validate it
  const filename = (await ask('Enter filename:')).trim();
  if (filename === '') {
    console.log('Error: Filename is required');
    return;
  }
  const fileExists = fs.existsSync(filename);

  // 2. Accept the content, then create / write the file
  const content = await ask('Enter content: ');
  if (content.trim() === '') {
    if (!fileExists) {
      console.log('Error: Content cannot be empty');
      return;
    }
    console.log('No new content entered. Existing file is kept as it is.');
```

```
  } else {
    fs.writeFileSync(filename, content);
```

```

    console.log(fileExists
      ? 'Existing file overwritten successfully.'
      : 'File created and content written successfully.');
```

```

  }

  // 3. Read the contents of the file
  console.log('\nContents of ' + filename + ':');
  console.log(fs.readFileSync(filename, 'utf8'));

  // 4. Accept additional content and append it
  const extra = await ask('\nEnter additional content to append (Enter to skip): ');
  if (extra.trim() !== '') {
    fs.appendFileSync(filename, extra);
    console.log('Additional content appended successfully.');
```

```

  }

  // 5. Display the final contents of the file
  console.log('\nFinal contents of ' + filename + ':');
  console.log(fs.readFileSync(filename, 'utf8'));
}

main()
  .catch(err => console.log('Error: ' + err.message))
  .finally(() => rl.close());
```

Run command: `node app.js`

7. CODE EXPLANATION

Tag / Statement	Description
<code>const fs = require('fs')</code>	Imports the built-in File System module used to create, read and append files.
<code>readline.createInterface({ input: process.stdin })</code>	Creates an interface that reads the text typed by the user from the keyboard (standard input).
<code>ask(question)</code>	Helper function that prints a prompt and returns the line entered by the user (used with <code>async/await</code>).
<code>filename.trim() === ''</code>	Validation: removes surrounding spaces and rejects an empty filename with “Filename is required”.
<code>fs.existsSync(filename)</code>	Checks whether the file already exists, so that the correct message is shown and an existing file can be overwritten or kept.
<code>content.trim() === ''</code>	Validation: rejects empty content with “Content cannot be empty” when the file does not already exist.
<code>fs.writeFileSync(filename, content)</code>	Creates the file and writes the content. If the file already exists, its old content is overwritten.
<code>fs.readFileSync(filename, 'utf8')</code>	Reads the whole file and returns its content as text, which is displayed using <code>console.log()</code> .
<code>fs.appendFileSync(filename, extra)</code>	Adds the additional content at the end of the file; the existing content remains unchanged.
<code>main().catch(...).finally(...)</code>	Runs the program, prints any file-system error message and always closes the readline interface.

8. TEST CASES AND EXECUTION DATA

The program was run from the command line (node app.js) and tested with the following filename and content combinations, each verified against the expected result:

Test Case	Input	Expected Output	Type	Status
TC-01	Filename: notes.txt Content: Hello Node.js	File is created and content is written successfully	✓ Valid	Pass
TC-02	Filename: student.txt Content: PBL Project	File is created with the given content	✓ Valid	Pass
TC-03	Filename: data.txt Content: Welcome Additional: " to Node.js"	Additional content is appended and final content is displayed	✓ Valid	Pass
TC-04	Filename: test.txt Content: (empty)	Display an error such as "Content cannot be empty"	✗ Invalid	Pass
TC-05	Filename: (empty) Content: Hello	Display "Filename is required"	✗ Invalid	Pass
TC-06	Filename: sample.txt Content: Hello	Existing file is overwritten if using writeFile / writeFileSync	✓ Valid	Pass
TC-07	Filename: sample.txt Additional: " World"	Existing content remains and " World" is appended	✓ Valid	Pass

Note: For TC-06 and TC-07, sample.txt already existed from an earlier run. In TC-07 the content prompt was left blank (Enter) so that the existing file is kept and only the additional content is appended. In TC-05 the program stops at the filename check, so the content prompt is not reached.

9. OUTPUT

Actual output produced by running the program above from the command line. TC-01 and TC-02 follow the same steps as TC-03 without the additional content, so TC-03 is shown as the complete flow:

```
●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename: data.txt
Enter content: Welcome
File created and content written successfully.

Contents of data.txt:
Welcome

Enter additional content to append (Enter to skip):  to Node.js
Additional content appended successfully.

Final contents of data.txt:
Welcome to Node.js
```

Fig. 9.1 — TC-03: file created, read, appended and final contents displayed (data.txt)

```
●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename: test.txt
Enter content:
Error: Content cannot be empty

●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename:
Error: Filename is required
```

Fig. 9.2 — TC-04 and TC-05: validation errors for empty content and empty filename

```
●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename: sample.txt
Enter content: Hello
Existing file overwritten successfully.

Contents of sample.txt:
Hello

Enter additional content to append (Enter to skip):

Final contents of sample.txt:
Hello

●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename: sample.txt
Enter content:
No new content entered. Existing file is kept as it is.

Contents of sample.txt:
Hello

Enter additional content to append (Enter to skip): World
Additional content appended successfully.

Final contents of sample.txt:
Hello World
```

Fig. 9.3 — TC-06 and TC-07: existing file overwritten, then appended (sample.txt)

10. RESULT

Thus, a Node.js application was successfully developed using the File System (fs) module. The program accepts a filename and content from the user and performs the file operations of creating/writing the file, reading its contents, appending additional content and displaying the final contents. Input validation for an empty filename and empty content was implemented and verified through command-line testing, matching the expected results shown above.

1. TITLE

Dynamic Modification of Content, Style and Attributes of HTML Elements Using the Document Object Model (DOM) and JavaScript

2. AIM

To explain the Document Object Model (DOM) and to develop a webpage that demonstrates how JavaScript can dynamically change the content, style and attributes of an HTML element without reloading the page.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the DOM as a tree representation of an HTML document.
- To select HTML elements using `document.getElementById()`.
- To change element content using `innerHTML`.
- To change element style using the `style` property.
- To change element attributes using `setAttribute()`, `getAttribute()` and `removeAttribute()`.
- To trigger JavaScript functions from buttons using the `onclick` event.

4. THEORY

The **Document Object Model (DOM)** is a programming interface provided by the browser for HTML documents. When a page loads, the browser parses the HTML and builds a tree of objects called nodes: the document is the root, every tag becomes an element node, every attribute becomes an attribute node and every piece of text becomes a text node. JavaScript can read and modify this tree, and the browser instantly re-renders the page to match. Hence the page can be changed without reloading.

```
document
|-- html
    |-- head --- title
    |-- body
        |-- div.container
            |-- h1#title --- "Welcome to DOM"
            |-- p#message --- "This is the original content."
            |-- div.buttons --- button, button, button
```

JavaScript modifies an element in three ways. **Content** is changed by assigning a new value to `element.innerHTML`. **Style** is changed through `element.style.property` (CSS names are written in camelCase, e.g. `backgroundColor`), which sets inline CSS. **Attributes** such as `title`, `class`, `href` or `data-*` are changed with `setAttribute(name, value)`, read with `getAttribute(name)` and deleted with `removeAttribute(name)`. Elements are first located with `document.getElementById()`.

5. ALGORITHM / PROCEDURE

1. Create three files: `index.html`, `style.css` and `script.js`.

2. In index.html, create a container with a heading (id="title"), a paragraph (id="message") and three buttons: Change Content, Change Style and Change Attribute.
3. Link style.css and script.js to the HTML page.
4. In style.css, style the body, container, title, message and buttons (including a hover effect).
5. In script.js, define changeContent() which uses getElementById() and innerHTML to replace the heading and paragraph text.
6. Define changeStyle() which uses the style property to change colour, font size, font family, background colour and padding.
7. Define changeAttribute() which uses setAttribute() to set the title and data-status attributes of the message paragraph, and getAttribute() to confirm the new value.
8. Attach each function to its button using the onclick attribute.
9. Open index.html in a browser and click each button, observing the change without page reload.
10. Test with different click sequences and record the observations against the expected results.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Manipulation</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1 id="title">Welcome to DOM</h1>
    <p id="message">This is the original content.</p>
    <div class="buttons">
      <button onclick="changeContent()">Change Content</button>
      <button onclick="changeStyle()">Change Style</button>
      <button onclick="changeAttribute()">Change Attribute</button>
    </div>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; }
body {
  margin: 0; min-height: 100vh;
  display: flex; justify-content: center; align-items: center;
  font-family: Arial, sans-serif;
  background: linear-gradient(135deg, #dbeafe, #f3e8ff);
}
.container {
  width: 600px; padding: 40px; text-align: center;
  background-color: white; border-radius: 20px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.15);
}
#title { color: #2563eb; font-size: 40px; margin-bottom: 25px; }
#message { padding: 15px; font-size: 18px; background-color: #f1f5f9;
  border-radius: 8px; margin-bottom: 25px; }
.buttons { display: flex; justify-content: center; gap: 15px; }
button {
  padding: 12px 22px; border: none; border-radius: 8px;
  background-color: #2563eb; color: white;
  font-size: 15px; cursor: pointer;
}
button:hover { background-color: #1d4ed8; }
```

script.js

```
// Function to change the content
function changeContent() {
  document.getElementById("title").innerHTML =
    "Content Changed Successfully!";
  document.getElementById("message").innerHTML =
    "The content has been changed using JavaScript.";
}

// Function to change the style
function changeStyle() {
  document.getElementById("title").style.color = "#7c3aed";
  document.getElementById("title").style.fontSize = "45px";
  document.getElementById("title").style.fontFamily = "Georgia, serif";
  document.getElementById("message").style.backgroundColor = "#fce7f3";
  document.getElementById("message").style.color = "#831843";
  document.getElementById("message").style.fontSize = "20px";
  document.getElementById("message").style.fontFamily = "Georgia, serif";
  document.getElementById("message").style.padding = "20px";
}

// Function to change the attributes
function changeAttribute() {
  var msg = document.getElementById("message");
  msg.setAttribute("title", "Attribute changed using JavaScript");
  msg.setAttribute("data-status", "updated");
  console.log(msg.getAttribute("title")); // read the new value
}
```

7. CODE EXPLANATION

Tag / Statement	Description
document.getElementById("title")	Selects the element with id "title" from the DOM so it can be modified.
.innerHTML = "..."	Replaces the content inside the selected element; the page updates instantly.
.style.color / .fontSize / .backgroundColor	Sets inline CSS properties of the element (CSS property names are written in camelCase).
.setAttribute("title", "...")	Adds a new attribute or changes its value if it already exists.
.getAttribute("title")	Reads the current value of an attribute.

Tag / Statement	Description
onclick="changeContent()"	Event attribute that calls the JavaScript function when the button is clicked.
display:flex; justify-content:center	CSS used to centre the container on the page and place the buttons in a row.
button:hover	CSS rule that darkens the button colour when the mouse is over it.

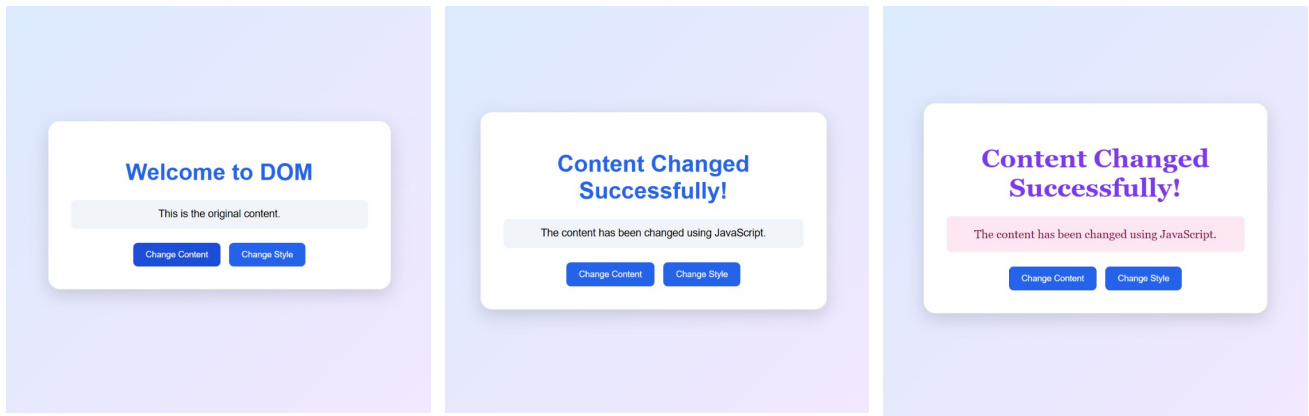
8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and the buttons were clicked in different sequences. Each observation was verified against the expected result:

Test Case	Action	Expected Result	Status
TC-01	Open the page (no click)	Title "Welcome to DOM" (blue, 40px); message "This is the original content." on light grey background	Pass
TC-02	Click Change Content	Title becomes "Content Changed Successfully!"; message becomes "The content has been changed using JavaScript."; styles unchanged	Pass
TC-03	Click Change Style	Title turns purple (#7c3aed), 45px, Georgia; message gets pink background (#fce7f3), dark red text, 20px, padding 20px	Pass
TC-04	Click Change Attribute	Message gets title = "Attribute changed using JavaScript" (tooltip on hover) and data-status = "updated"; visible text unchanged	Pass
TC-05	Click Change Content, then Change Style	Both the new text and the new style are shown together	Pass
TC-06	Click Change Style, then Change Content	Text changes but the purple/Georgia style is retained (innerHTML changes only content)	Pass
TC-07	Click Change Content twice	Same result as one click; content is replaced, not duplicated	Pass
TC-08	Click Change Attribute twice	Attribute values are overwritten, not duplicated	Pass
TC-09	Hover the mouse over a button	Button colour changes from #2563eb to #1d4ed8	Pass
TC-10	Refresh the page after any change	All changes are lost; the page returns to the original state (DOM changes are not permanent)	Pass

9. OUTPUT

Actual output produced by running the program in a browser: (a) original page, (b) after clicking Change Content, (c) after clicking Change Style.



(a) Original page

(b) Change Content

(c) Change Style

Fig. 9.1 — DOM manipulation: original page, changed content and changed style

10. RESULT

Thus, the Document Object Model (DOM) was studied and a webpage was successfully developed using HTML, CSS and JavaScript. JavaScript functions were used to dynamically change the content (innerHTML), style (style property) and attributes (setAttribute) of HTML elements without reloading the page, and the behaviour was verified through browser testing, matching the expected results shown above.

1. TITLE

Express.js Student Server: Creating a Basic Web Server with Routes (/, /students, /about) Using Node.js and Express.js

2. AIM

To develop a basic Express.js server with the routes /, **/students** and **/about**, where the /students route returns a list of at least five students in JSON format and the /about route displays information about the application, using appropriate HTTP methods and routing.

3. OBJECTIVE

The objectives of this experiment are:

- To understand Express.js as a web framework built on Node.js.
- To create a server using `express()` and start it with `app.listen()`.
- To define routes using the HTTP GET method with `app.get()`.
- To send HTML responses using `res.send()` and JSON responses using `res.json()`.
- To store student data as an array of objects and return it from a route.
- To handle invalid URLs using a fallback middleware that returns the status code 404.

4. THEORY

Express.js is a minimal and flexible web application framework for Node.js. It simplifies creating servers and handling HTTP requests. The framework is installed with `npm install express` and loaded using `require("express")`. Calling `express()` returns an application object (usually named `app`) used to define routes and start the server.

Routing means deciding how the application responds to a client request for a particular URL (path) and HTTP method. A route has the form `app.METHOD(path, handler)`, for example `app.get("/students", (req, res) => {...})`. The handler receives the request object `req` and the response object `res`. The GET method is used here because the routes only retrieve data (POST would create, PUT update and DELETE remove data).

The response is sent using `res.send()` (text or HTML) or `res.json()` (converts a JavaScript object/array to JSON and sets the header `Content-Type: application/json`). Routes are checked in the order in which they are written, so a middleware written with `app.use()` after all the routes runs only when no route matches; it is used here to return the status 404 (Not Found). Finally, `app.listen(PORT, callback)` starts the server on the given port (3000).

5. ALGORITHM / PROCEDURE

1. Create a project folder, open a terminal in it and run `npm init -y` to create `package.json`.
2. Install Express using `npm install express`.
3. Create a file named `app.js` and import `express`; create the `app` object and set `PORT = 3000`.
4. Create an array named `students` containing five student objects (`id`, `name`, `branch`).
5. Define the GET route `"/` that sends a heading and a welcome message.
6. Define the GET route `"/students` that returns the `students` array using `res.json()`.

7. Define the GET route `"/about"` that sends information about the application.
8. Define a fallback middleware using `app.use()` that sends status 404 with the message `"404 - Page Not Found"` for any other URL.
9. Start the server using `app.listen()` and display the server URL on the console.
10. Run the program using `node app.js` and open `http://localhost:3000` in a browser.
11. Visit each route, test invalid URLs and record the observations against the expected results.

6. PROGRAM

app.js

```
const express = require("express");
const app = express();
const PORT = 3000;

// Student data
const students = [
  { id: 1, name: "Arun", branch: "CSE" },
  { id: 2, name: "Priya", branch: "AIML" },
  { id: 3, name: "Rahul", branch: "ECE" },
  { id: 4, name: "Sita", branch: "CSE" },
  { id: 5, name: "Kiran", branch: "AIML" }
];

// Home route
app.get("/", (req, res) => {
  res.send(`
    <h1>Student Management Server</h1>
    <p>Welcome to the Express.js Student Application</p>
  `);
});

// Students route
app.get("/students", (req, res) => {
  res.json(students);
});

// About route
app.get("/about", (req, res) => {
  res.send(`
    <h1>About Application</h1>
    <p>This application is developed using Node.js and Express.js.</p>
    <p>It provides student information through different routes.</p>
  `);
});

// Invalid route
app.use((req, res) => {
  res.status(404).send("404 - Page Not Found");
});

// Start server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Run command: `node app.js`

7. CODE EXPLANATION

Tag / Statement	Description
<code>const express = require("express")</code>	Imports the Express.js framework installed through npm.
<code>const app = express()</code>	Creates the Express application object used to define routes and start the server.

Tag / Statement	Description
<code>const students = [...]</code>	Array of five objects; each has id, name and branch (the student data returned by /students).
<code>app.get("/", (req, res) => {...})</code>	Route for the home page; handles GET requests to "/" and sends an HTML heading and welcome message using <code>res.send()</code> .
<code>app.get("/students", ...)</code>	Route that returns the students array as JSON using <code>res.json()</code> .
<code>app.get("/about", ...)</code>	Route that sends an HTML page describing the application.
<code>app.use((req, res) => {...})</code>	Fallback middleware placed after all routes; runs only for unmatched URLs and returns <code>res.status(404).send(...)</code> .
<code>app.listen(PORT, callback)</code>	Starts the server on port 3000 and prints the URL on the console when it is ready.

8. TEST CASES AND EXECUTION DATA

The server was started using `node app.js` and the routes were requested from the browser (and with other HTTP methods where indicated). Each observation was verified against the expected result:

Test Case	Request	Expected Result	Status
TC-01	GET /	Heading "Student Management Server" and text "Welcome to the Express.js Student Application"	Pass
TC-02	GET /students	JSON array containing five students: Arun (CSE), Priya (AIML), Rahul (ECE), Sita (CSE), Kiran (AIML)	Pass
TC-03	GET /students (check response)	Content-Type is application/json; every object has id, name and branch; total 5 records	Pass
TC-04	GET /about	Page "About Application" describing that it is developed using Node.js and Express.js and provides student information	Pass
TC-05	GET /contact (undefined route)	Status 404 with message "404 - Page Not Found"	Pass
TC-06	POST /students	Status 404 (only the GET method is defined for this route)	Pass
TC-07	GET /students/	Trailing slash is ignored by default; the same student list is returned	Pass
TC-08	GET /Students	Route matching is case-insensitive by default; the same student list is returned	Pass
TC-09	Run <code>node app.js</code>	Console displays "Server running at http://localhost:3000"	Pass

9. OUTPUT

Actual output produced by running the program: the console message, the home page (route /) and the JSON list returned by the /students route.

```
$ node app.js
Server running at http://localhost:3000
```

Fig. 9.1 — Server started on port 3000

Student Management Server

Welcome to the Express.js Student Application

Fig. 9.2 — Route "/" (home page)

```
Pretty-print ☐  
[{"id":1,"name":"Arun","branch":"CSE"}, {"id":2,"name":"Priya","branch":"AIML"}, {"id":3,"name":"Rahul","branch":"ECE"}, {"id":4,"name":"Sita","branch":"CSE"}, {"id":5,"name":"Kiran","branch":"AIML"}]
```

Fig. 9.3 — Route "/students" returning five student records in JSON format

10. RESULT

Thus, a basic Express.js server was successfully developed using Node.js. The routes "/", "/students" and "/about" were created using the GET method: the /students route returns a list of five students in JSON format, the /about route displays information about the application, and any undefined URL returns a 404 error message. The behaviour was verified through browser testing, matching the expected results shown above.

PART A — Q5

Differentiating JavaScript Functions and Classes, and Creating Multiple Objects Using a Class

1. AIM

To differentiate between JavaScript functions and classes, and to demonstrate, with a working program, how a class can be used to create multiple objects that share common properties and methods.

2. OBJECTIVE

- To understand a regular JavaScript function and how it can be used as a constructor function.
- To understand the class syntax (ES6) as syntactic sugar over JavaScript's prototype-based inheritance.
- To compare functions and classes on definition, invocation, hoisting, strict mode and constructor rules.
- To create multiple objects from the same class, each with its own property values but sharing the same methods.
- To verify, through code, how a class enforces object creation only through the 'new' keyword.

3. THEORY

A JavaScript function is a general-purpose, reusable block of code. It can be called directly (`greet()`), used as a constructor with `new` (`new Student()`), or used as a method, a callback or an IIFE. Because of this flexibility, the value of `this` inside a normal function depends on how it is called, and calling a constructor function without `new` does not raise an error — it silently attaches properties to the wrong object (the global object, or `undefined` in strict mode).

A JavaScript class (introduced in ES6) is a template used specifically to create objects with a shared shape. It is defined with the `class` keyword, contains a special `constructor()` method that runs once per object to set initial property values, and any other methods written inside the class body are placed on the class's prototype and shared by every object created from it — they are not copied into each object individually. A class can only be instantiated with `new`; calling it as a plain function (`StudentClass()`) throws a `TypeError`. Class bodies also always execute in strict mode and class declarations are not hoisted the way function declarations are.

The key difference for object creation is this: a function only becomes a way of making objects when it is deliberately used as a constructor (by convention, given a capitalised name and called with `new`), whereas a class exists only to serve as a blueprint for objects. In both cases, methods defined on the prototype (via `Function.prototype.method = ...` for a function, or directly inside a class body) are stored once in memory and shared by every instance, while each instance keeps its own independent copy of the properties set inside the constructor.

3.1. Functions vs Classes — Comparison

Aspect	Function (used as constructor)	Class
Keyword	<code>function</code>	<code>class</code>
Purpose	General-purpose; can also be a constructor by convention	Dedicated blueprint for creating objects
Invocation	Can be called with or without 'new'	Must always be called with 'new'
Calling without 'new'	No error; 'this' binds incorrectly (global object / undefined)	Throws: 'Class constructor ... cannot be invoked without new'
Hoisting	Function declarations are hoisted (usable before definition)	Class declarations are not initialised until evaluated (temporal dead zone)

Aspect	Function (used as constructor)	Class
Strict mode	Not strict unless declared	Body always runs in strict mode automatically
Methods	Added manually via <code>Function.prototype.method = ...</code>	Written directly inside the class body; auto-added to the prototype
Inheritance	Manual prototype chaining (<code>Object.create</code> , <code>.call()</code>)	Built-in 'extends' and 'super' keywords
Re-declaration	Can be re-declared / reassigned	Class name in the same scope cannot be re-declared

How a class produces multiple objects: every time `new StudentClass(name, branch)` is executed, JavaScript creates a brand-new object, runs the `constructor()` with that new object as `this`, and links the object's internal prototype to `StudentClass.prototype`. Each resulting object therefore has its own independent name and branch values, while all of them share the single `display()` method that lives on the prototype — the method is not duplicated for every object, which saves memory as more objects are created.

4. ALGORITHM / PROCEDURE

- Create a file named `app.js`.
- Define a constructor function `StudentFunction(name, branch)` that sets `this.name` and `this.branch`, and add a `display()` method to `StudentFunction.prototype`.
- Define an equivalent class `StudentClass` with a `constructor(name, branch)` and a `display()` method written directly inside the class body.
- Create two objects using `'new StudentFunction(...)'` and call `display()` on each.
- Create three objects using `'new StudentClass(...)'` and call `display()` on each, to show that a class can produce any number of independent objects that share the same method.
- Use `typeof` on both `StudentFunction` and `StudentClass` to show both are technically of type `'function'`.
- Call `StudentClass(...)` without `'new'` inside a `try/catch` block and print the resulting error, to show that a class enforces the use of `'new'`.
- Call `StudentFunction(...)` without `'new'` to show that, unlike a class, no error is raised, and observe that `'this'` inside the call is bound incorrectly.
- Save the file and run it using: `node app.js`
- Record the console output and compare it against the expected behaviour.

5. PROGRAM

`app.js`

```
// Comparing JavaScript Functions and Classes

// 1. Object creation using a constructor FUNCTION
function StudentFunction(name, branch) {
  this.name = name;
  this.branch = branch;
}
StudentFunction.prototype.display = function () {
  console.log(this.name + " - " + this.branch);
};

// 2. Object creation using a CLASS
class StudentClass {
  constructor(name, branch) {
    this.name = name;
    this.branch = branch;
  }
  display() {
    console.log(this.name + " - " + this.branch);
  }
}
console.log("--- Using constructor function ---");
```

```

const f1 = new StudentFunction("Arun", "CSE");
const f2 = new StudentFunction("Priya", "AIML");
f1.display();
f2.display();

console.log("\n--- Using class ---");
const c1 = new StudentClass("Rahul", "ECE");
const c2 = new StudentClass("Sita", "CSE");
const c3 = new StudentClass("Kiran", "AIML");
c1.display();
c2.display();
c3.display();

console.log("\n--- typeof check ---");
console.log("typeof StudentFunction:", typeof StudentFunction);
console.log("typeof StudentClass:", typeof StudentClass);

console.log("\n--- Calling class without 'new' ---");
try {
  StudentClass("Test", "IT");
} catch (err) {
  console.log("Error:", err.message);
}

console.log("\n--- Calling function without 'new' (no error, but 'this' is wrong) ---");
StudentFunction("Test", "IT");
Run command: node app.js

```

6. CODE EXPLANATION

Tag / Statement	Description
function StudentFunction(name, branch) {...}	A regular function used as a constructor; 'this' refers to the new object only when called with 'new'.
StudentFunction.prototype.display = ...	Manually adds a shared method to the function's prototype so every instance can use it without copying it.
class StudentClass { constructor(...) {...} display() {...} }	Defines a class with a constructor (runs per object) and a display() method (added automatically to the class's prototype and shared by all instances).
new StudentFunction(...) / new StudentClass(...)	Creates a new object, runs the constructor/function with 'this' bound to that object, and returns it.
typeof StudentFunction / typeof StudentClass	Both evaluate to 'function', showing that a class is implemented on top of JavaScript's function/prototype system.
try { StudentClass(...) } catch (err) {...}	Calling a class without 'new' throws a TypeError; the try/catch captures and prints that error message.
StudentFunction('Test', 'IT') (no 'new')	Demonstrates that, unlike a class, calling a constructor function without 'new' does not throw — it silently misassigns 'this', a common source of bugs.

7. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Status
TC-01	Create f1, f2 with 'new StudentFunction(...)' and call display()	Arun - CSE and Priya - AIML printed, one line each	Pass
TC-02	Create c1, c2, c3 with 'new StudentClass(...)' and call display()	Rahul - ECE, Sita - CSE and Kiran - AIML printed; three independent objects share one display() method	Pass
TC-03	typeof StudentFunction and typeof StudentClass	Both print 'function'	Pass

Test Case	Action	Expected Result	Status
TC-04	Call StudentClass('Test','IT') without 'new'	TypeError thrown: "Class constructor StudentClass cannot be invoked without 'new'"	Pass
TC-05	Call StudentFunction('Test','IT') without 'new'	No error is thrown; the function runs but 'this' does not refer to a Student object	Pass

8. OUTPUT

Actual console output produced by running the program above with node app.js:

```
$ node app.js
--- Using constructor function ---
Arun - CSE
Priya - AIML

--- Using class ---
Rahul - ECE
Sita - CSE
Kiran - AIML

--- typeof check ---
typeof StudentFunction: function
typeof StudentClass: function

--- Calling class without 'new' ---
Error: Class constructor StudentClass cannot be invoked without 'new'

--- Calling function without 'new' (no error, but 'this' is wrong) ---
```

Fig. 8.1 — Console output comparing constructor-function and class object creation

9. RESULT

Thus, JavaScript functions and classes were studied and compared. It was observed that a function only creates objects when deliberately used as a constructor with 'new', whereas a class is a dedicated blueprint that always requires 'new' and throws an error otherwise. A class (StudentClass) was used to create three independent objects (Rahul, Sita, Kiran), each holding its own name and branch while sharing a single display() method defined on the prototype, confirming that a class allows multiple objects to be created efficiently with common properties and methods. The behaviour was verified through command-line testing, matching the expected results shown above.

PART A — Q6

Express.js Middleware for Logging Incoming Requests

1. AIM

To explain the concept of middleware in Express.js, describe how it works within the request-response cycle, and develop an example of logging middleware that records the HTTP method and URL of every incoming request.

2. OBJECTIVE

- To understand what middleware is in Express.js and its function signature (req, res, next).
- To understand how middleware fits into the request-response cycle and the role of the next() function.
- To understand the difference between application-level middleware and route handlers.
- To develop a logging middleware that records the method and URL of every incoming request.
- To verify, through testing, that the middleware executes before the route handler for every route.

3. THEORY

Middleware in Express.js is a function that has access to the request object (req), the response object (res), and a third argument, next, which is a function used to pass control to the next middleware or route handler in the pipeline. Its general signature is function(req, res, next) { ... }.

A middleware function can perform any of the following: execute code such as logging, parsing or authentication; modify the req or res objects; end the request-response cycle by sending a response; or call next() to pass control onward. Middleware is registered using app.use() (applied to every request) or attached to a specific route as an extra argument before the route handler.

Middleware in the request-response cycle: When a request arrives at an Express server, it passes through the registered middleware functions in the exact order in which they were defined, forming a pipeline: request → middleware 1 → middleware 2 → ... → matching route handler → response. Each middleware must either call next() to hand control to the next function in the chain, or send a response itself (using res.send(), res.json(), res.status().send(), etc.), which ends the cycle at that point. If a middleware neither calls next() nor sends a response, the request hangs and the client never receives a reply. Because middleware registered with app.use() runs before the routes defined after it, it is commonly placed at the top of the file so that it runs for every incoming request, regardless of which route is ultimately matched.

A logging middleware is a simple example of this pattern: it does not send a response itself; it only records information about the request (such as the HTTP method and URL) and then calls next() so that the actual route handler can process the request and send the response.

4. ALGORITHM / PROCEDURE

1. Create a project folder and a file named app.js.
2. Import Express using require("express") and create the application object with express().
3. Define a middleware function using app.use() that runs for every incoming request, before any route is declared.
4. Inside the middleware, read req.method and req.url and print them to the console using console.log().
5. Call next() at the end of the middleware so that control passes on to the matching route handler.
6. Define GET routes for "/", "/student" and "/about", each sending a simple text response using res.send().
7. Start the server on port 3000 using app.listen() and display a confirmation message on the console.
8. Run the program using the command: node app.js
9. Open each route in the browser and observe the method and URL logged on the terminal for every request.
10. Test the middleware with different routes and record the observations against the expected results.

5. PROGRAM

middleware.js

```
const express = require("express");
const app = express();
```

```
// Middleware
app.use((req, res, next) => {
  console.log("Request Method:", req.method);
  console.log("Request URL:", req.url);
  next();
});

// Home route
app.get("/", (req, res) => {
  res.send("Welcome to Middleware Demo");
});

// Student route
app.get("/student", (req, res) => {
  res.send("Student details displayed");
});

// About route
app.get("/about", (req, res) => {
  res.send("This is the About page");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
Run command: node middleware.js
```

6. CODE EXPLANATION

Tag / Statement	Description
const express = require("express")	Imports the Express.js framework and creates the app object used to define middleware and routes.
app.use((req, res, next) => {...})	Registers application-level middleware that runs for every incoming request, before any route handler.
req.method, req.url	Properties of the request object that give the HTTP method (GET, POST, etc.) and the requested URL path.
console.log(...)	Prints the method and URL of the incoming request to the terminal, forming the log entry.
next()	Passes control from the middleware to the next matching function in the pipeline (here, the route handler); without it the request would never reach a route.
app.get("/", ...), app.get("/student", ...), app.get("/about", ...)	Route handlers that run after the logging middleware and send the final response using res.send().
app.listen(3000, ...)	Starts the server on port 3000 and prints a confirmation message once it is ready.

7. TEST CASES AND EXECUTION DATA

Test Case	Request	Expected Console Log	Status
TC-01	GET /	Request Method: GET, Request URL: /	Pass
TC-02	GET /student	Request Method: GET, Request URL: /student	Pass
TC-03	GET /about	Request Method: GET, Request URL: /about	Pass
TC-04	GET /unknown	Request Method: GET, Request URL: /unknown logged, then Express's default "Cannot GET /unknown" is sent (no matching route)	Pass
TC-05	Multiple requests in sequence	A separate log entry is printed for every request, in the order the requests arrive	Pass

8. OUTPUT

Actual output produced by running the program above (server started and a GET request made to the home route "/"), as shown in the browser and the integrated terminal:

Browser window: a page at <http://localhost:3000/> displaying "Welcome to Middleware Demo".

```
PS C:\Users\aruna\OneDrive\Desktop\fullstack> node middleware.js
Server running on port 3000
Request Method: GET
Request URL: /
```

Fig. 8.1 — Middleware logging the method and URL of the incoming GET / request

9. RESULT

Thus, the concept of middleware in Express.js was studied and its role in the request-response cycle was explained. A logging middleware was successfully developed using `app.use()`, which records the HTTP method and URL of every incoming request to the console and then calls `next()` to pass control to the appropriate route handler. The behaviour was verified through browser testing across multiple routes, matching the expected results shown above.

PART A — Q7

Node.js Modules: Using the os, path and fs Built-in Modules

1. AIM

To explain what Node.js modules are, and to demonstrate, with a working program, the purpose and usage of three core built-in modules — os, path and fs — by retrieving system information, working with file paths, and writing/reading a file.

2. OBJECTIVE

- To understand what a module is in Node.js and how modules are loaded using require().
- To distinguish between built-in (core), local, and third-party modules.
- To use the os module to retrieve operating system and hardware information.
- To use the path module to construct and inspect file paths in a platform-independent way.
- To use the fs module to write to and read from a file synchronously.

3. THEORY

A module in Node.js is a reusable, self-contained block of JavaScript code, typically stored in its own file, that can be loaded into another file using the require() function. Node.js wraps every file in a module wrapper function, which gives each file its own scope so that variables and functions defined in one file do not leak into another unless they are explicitly exported with module.exports. Modules make code organised, reusable and easier to maintain, since related functionality can be grouped together and imported wherever it is needed.

Node.js modules fall into three categories: built-in (core) modules, which ship with Node.js itself and require no installation (e.g. os, path, fs, http); local modules, which are files created within the project and imported using a relative path (e.g. require("./myModule")); and third-party modules, which are installed from npm and imported by package name (e.g. require("express")).

The **os** module provides information about the operating system and the underlying hardware the Node.js process is running on. Commonly used methods include os.platform() (the operating system platform, e.g. "win32", "linux", "darwin"), os.arch() (the CPU architecture, e.g. "x64"), os.totalmem() and os.freemem() (total and free system memory in bytes), and os.cpus() (details of each logical CPU core). It is useful for writing code that needs to adapt to, or report on, the environment it is running in.

The **path** module provides utilities for working with file and directory paths in a way that is consistent across operating systems (Windows uses backslashes, while Linux/macOS use forward slashes). Commonly used methods include path.join() (safely joins path segments into one path), path.basename() (returns the last portion of a path, i.e. the file name), path.dirname() (returns the directory portion of a path), and path.extname() (returns the file's extension). Using path.join() with __dirname (the absolute path of the current file's directory) is the standard way to build reliable, portable file paths.

The **fs** (File System) module allows a Node.js program to interact with the file system — creating, reading, writing, appending to and deleting files and directories. Commonly used synchronous methods include fs.writeFileSync(path, data) (creates a file or overwrites it if it exists), fs.readFileSync(path, 'utf8') (reads the entire file and returns its contents as a string), fs.appendFileSync(path, data) (adds data to the end of an existing file), and fs.existsSync(path) (checks whether a file or directory exists). Asynchronous counterparts (fs.writeFile, fs.readFile, etc.) perform the same operations without blocking the rest of the program, using callbacks or promises.

Together, these three modules show how Node.js's built-in modules let a program inspect its environment (os), build correct file paths regardless of platform (path), and persist data to disk (fs) — all without installing any external package.

4. ALGORITHM / PROCEDURE

1. Create a project folder and a file named app.js.
2. Import the built-in os, path and fs modules using require().
3. Use os.platform(), os.arch() and os.totalmem() to print the operating system, CPU architecture and total memory.

4. Use `path.join(__dirname, "student.txt")` to build the full path of a file inside the project folder.
5. Use `path.basename()`, `path.dirname()` and `path.extname()` on that path to print the file name, directory and extension.
6. Use `fs.writeFileSync()` to create/overwrite `student.txt` with sample text content.
7. Use `fs.readFileSync()` to read the file back and print its content to the console.
8. Save the file and run it using the command: `node app.js`
9. Observe the console output and verify that `student.txt` was created in the project folder with the expected content.

5. PROGRAM

app.js

```
const os = require("os");
const path = require("path");
const fs = require("fs");

// OS Module
console.log("Operating System:", os.platform());
console.log("CPU Architecture:", os.arch());
console.log("Total Memory:", os.totalmem());

// Path Module
const filePath = path.join(__dirname, "student.txt");
console.log("File Path:", filePath);
console.log("File Name:", path.basename(filePath));
console.log("Directory:", path.dirname(filePath));
console.log("Extension:", path.extname(filePath));

// FS Module
fs.writeFileSync(filePath, "Welcome to Node.js");
const data = fs.readFileSync(filePath, "utf8");
console.log("File Content:", data);
Run command: node app.js
```

6. CODE EXPLANATION

Tag / Statement	Description
<code>const os = require("os")</code>	Imports the built-in <code>os</code> module, giving access to operating system and hardware information.
<code>const path = require("path")</code>	Imports the built-in <code>path</code> module, used to build and inspect file paths in a platform-independent way.
<code>const fs = require("fs")</code>	Imports the built-in <code>File System</code> module, used to write to and read from files on disk.
<code>os.platform(), os.arch(), os.totalmem()</code>	Return the operating system platform, the CPU architecture and the total system memory in bytes.
<code>path.join(__dirname, "student.txt")</code>	Builds the absolute path of <code>student.txt</code> inside the current directory, joining segments correctly for the current OS.
<code>path.basename() / path.dirname() / path.extname()</code>	Extract the file name, the containing directory and the file extension from the full path.
<code>fs.writeFileSync(filePath, "Welcome to Node.js")</code>	Creates <code>student.txt</code> (or overwrites it if it already exists) with the given text content.
<code>fs.readFileSync(filePath, "utf8")</code>	Reads the full content of <code>student.txt</code> as a UTF-8 string.
<code>console.log("File Content:", data)</code>	Prints the content that was just read back from the file, confirming the write/read cycle worked.

7. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Status
TC-01	os.platform(), os.arch()	Prints the current OS platform (e.g. win32) and CPU architecture (e.g. x64)	Pass
TC-02	os.totalmem()	Prints total system memory in bytes as a positive number	Pass
TC-03	path.join(__dirname, "student.txt")	Prints the correct absolute path of student.txt inside the project folder	Pass
TC-04	path.basename() / dirname() / extname()	Prints student.txt, the project directory path, and .txt respectively	Pass
TC-05	fs.writeFileSync() then fs.readFileSync()	student.txt is created with the text "Welcome to Node.js" and the same text is read back and printed	Pass
TC-06	Run node app.js a second time	student.txt is overwritten with the same content; output remains identical (no error)	Pass

8. OUTPUT

Actual output produced by running the program above from the command line:

```
PS C:\Users\aruna\OneDrive\Desktop\fullstack> node app.js
Operating System: win32
CPU Architecture: x64
Total Memory: 16407719936
File Path: C:\Users\aruna\OneDrive\Desktop\fullstack\student.txt
File Name: student.txt
Directory: C:\Users\aruna\OneDrive\Desktop\fullstack
Extension: .txt
File Content: Welcome to Node.js
PS C:\Users\aruna\OneDrive\Desktop\fullstack>
```

Fig. 8.1 — Console output showing OS details, path breakdown and file content written/read using os, path and fs

9. RESULT

Thus, Node.js modules were studied, along with the purpose of the built-in os, path and fs modules. The os module was used to display the operating system, CPU architecture and total memory; the path module was used to build and break down a file path into its name, directory and extension; and the fs module was used to write content to student.txt and read it back successfully. The behaviour was verified through command-line testing, matching the expected results shown above.

PART A — Q8

NPM and package.json in Node.js: Installing and Using an External Package

1. AIM

To explain NPM (Node Package Manager) and the role of package.json in a Node.js application, and to describe and demonstrate the steps involved in installing an external package and using it in a working Node.js program.

2. OBJECTIVE

- To understand what NPM is and why it is used in Node.js development.
- To understand the purpose and structure of the package.json file.
- To learn the steps involved in initialising a Node.js project and installing an external package.
- To demonstrate the use of an installed external package (chalk) inside a Node.js application.

3. THEORY

NPM (Node Package Manager) is the default package manager that is bundled with Node.js. It is used to install, share, update and manage reusable pieces of code, called packages or modules, that other developers have published. Instead of writing every piece of functionality from scratch, a developer can search the NPM registry for a package that already solves the problem and install it into the project with a single command. NPM also manages the versions of these packages and keeps track of which versions a project depends on, so that the same project can be set up identically on any other machine.

package.json is a JSON file kept at the root of every Node.js project. It acts as the manifest, or identity card, of the project. It stores metadata such as the project's name, version, description, entry point (main file), author and license, and, most importantly, it lists the project's dependencies (external packages required to run the project) and devDependencies (packages required only during development, such as testing tools). When a package is installed with the `--save` behaviour (the default in modern NPM), its name and version range are automatically added to the dependencies section of package.json. This allows anyone who receives the project folder to recreate the exact same set of packages simply by running `'npm install'`, without needing to send the actual package files.

Whenever a package is installed, NPM downloads it into a local folder named `node_modules` inside the project, and records the exact resolved version in a companion file called `package-lock.json`, which ensures that every install produces the same dependency versions. The `node_modules` folder and the lock file together with package.json form the complete dependency management system of a Node.js project.

4. STEPS TO INSTALL AND USE AN EXTERNAL PACKAGE

1. Create a project folder and open a terminal inside it.
2. Run `npm init -y` to generate a default package.json file (the `-y` flag accepts all default values automatically).
3. Run `npm install chalk` (or the shorthand `npm i chalk`) to download the 'chalk' package from the NPM registry. NPM creates the `node_modules` folder, downloads the package into it, and adds an entry for 'chalk' under "dependencies" in package.json.
4. Import the package inside the application file using `const chalk = require("chalk");`
5. Use the functions/methods exposed by the package (for example `chalk.blue()`, `chalk.green()`, `chalk.red()`) anywhere in the program to use its functionality.
6. Run the application using `node app.js`; Node.js resolves the `'require("chalk")'` call by looking inside the local `node_modules` folder.

5. PROGRAM

Step 1 — Initialise the project and install the package (terminal):

```
$ npm init -y
$ npm install chalk
```

Step 2 — app.js (Student Management System using the 'chalk' package):

```
const chalk = require("chalk");

// Student details
```

```

const students = [
  { name: "Aruna",    rollNo: 101, branch: "AI & ML", marks: 85 },
  { name: "Priya",    rollNo: 102, branch: "CSE",      marks: 92 },
  { name: "Divya",    rollNo: 103, branch: "ECE",      marks: 78 },
  { name: "Keerthi",  rollNo: 104, branch: "IT",       marks: 88 }
];

// Heading
console.log(chalk.blue("====="));
console.log(chalk.green("      STUDENT MANAGEMENT SYSTEM"));
console.log(chalk.blue("====="));
console.log();

// Display students
console.log(chalk.yellow("Student Details"));
console.log("-----");

students.forEach((student) => {
  console.log(chalk.cyan("Name      : ") + student.name);
  console.log(chalk.cyan("Roll No  : ") + student.rollNo);
  console.log(chalk.cyan("Branch   : ") + student.branch);
  console.log(chalk.cyan("Marks    : ") + student.marks);

  // Grade calculation
  let grade;
  if (student.marks >= 90) grade = "A+";
  else if (student.marks >= 80) grade = "A";
  else if (student.marks >= 70) grade = "B";
  else if (student.marks >= 60) grade = "C";
  else grade = "Fail";

  if (grade === "Fail") {
    console.log(chalk.red("Grade      : " + grade));
  } else {
    console.log(chalk.green("Grade      : " + grade));
  }
  console.log("-----");
});

// Calculate average
let total = 0;
students.forEach((student) => { total = total + student.marks; });
const average = total / students.length;

console.log(chalk.magenta("Total Students : " + students.length));
console.log(chalk.magenta("Total Marks    : " + total));
console.log(chalk.magenta("Average Marks  : " + average.toFixed(2)));

console.log();
console.log(chalk.green("Program executed successfully!"));
console.log(chalk.blue("NPM external package 'chalk' was used."));
Run command: node app.js

```

6. CODE EXPLANATION

Tag / Statement	Description
npm init -y	Generates package.json automatically with default project metadata.
npm install chalk	Downloads the 'chalk' package into node_modules and records it under "dependencies" in package.json.
const chalk = require("chalk");	Imports the installed package so its functions can be used in app.js.
chalk.blue(), chalk.green(), chalk.red(), etc.	Package functions that wrap a string with ANSI colour codes so it prints in colour on the terminal.
students.forEach(...)	Iterates over the array of student objects to print each student's details.
if / else if grade calculation	Assigns a letter grade to each student based on the marks scored.

Tag / Statement	Description
total / average calculation	Accumulates total marks and computes the class average using a <code>forEach</code> loop.

7. OUTPUT

Running the program with `node app.js` produces a colour-formatted console report: a blue-and-green banner, each student's Name, Roll No, Branch and Marks in cyan, a grade in green (or red for a fail), followed by the total students, total marks and average marks in magenta, and a final success message.

Sample output (colours rendered as plain text): Aruna (101, AI & ML, 85, Grade A), Priya (102, CSE, 92, Grade A+), Divya (103, ECE, 78, Grade B), Keerthi (104, IT, 88, Grade A); Total Students: 4, Total Marks: 343, Average Marks: 85.75, followed by "Program executed successfully!" and "NPM external package 'chalk' was used."

8. RESULT

Thus, NPM and `package.json` were studied and explained. NPM was understood as Node.js's package manager used to install and manage external packages, and `package.json` was understood as the project manifest that records project metadata and dependencies. The steps to install an external package (`npm init`, `npm install`, `require`, `use`) were demonstrated by installing the 'chalk' package and using it to build a colour-formatted Student Management System, confirming that an external NPM package can be installed and used successfully in a Node.js application.